

Bilaga A

Övningstentamen - C++-teori

Information

Hjälpmedel

- Papper och penna.

Poänggränser och betygsnivåer

Totalt: 20 poäng.

Betygsgränser:

- **G:** Minst 10 poäng.
- **VG:** Minst 15 poäng.

Bidrag till kursens slutpoäng:

- Betyget **G** ger 2 poäng till kurssammanställningen.
- Betyget **VG** ger 4 poäng till kurssammanställningen.

Viktigt

- All kod ska implementeras i en headerfil, metoder definieras i klassen.
- Kompileringsdirektiv såsom `#pragma once` behövs ej!
- Inkludering av headerfiler behövs bara för standardheaders!

G-uppgifter

1. Interface (2p)

Skapa ett nytt interface `driver::gpio::Interface`, som innehåller följande virtuella metoder:

- Destruktor, ska markeras `virtual` och sättas till `= default`.
- `read()`, som returnerar GPIO-instansens tillstånd (högt/lågt) som `true/false`.
- `write(bool state)`, som sätter GPIO-instansens tillstånd (`true` = högt).
- `toggle()`, som togglar GPIO-instansens tillstånd.

Viktigt:

- Samtliga metoder ska markeras `noexcept`.
 - Samtliga metoder (förutom destruktorn) ska deklarerars som rent virtuella (`= 0`).
 - `read()` ska vara `const`.
-

2. Stubbklass (4p)

Skapa en underklass `driver::gpio::Stub`, som ärver ovanstående interface.

Implementera denna klass till en enkel stubb, som möjliggör att man kan sätta GPIO-instansens tillstånd via en privat medlemsvariabel `myState`. Via denna variabel ska man kunna läsa, skriva och toggla GPIO-instansen.

Denna klass ska innehålla:

- En privat medlemsvariabel `myState`, som sparar GPIO-instansens tillstånd (hög/låg).
- En default-konstruktor, som initierar `myState` till `false`.
- En destruktor, som sätts till `= default`.
- Överlagrade varianter av metoderna från interfacet:
 - `read()` returnerar `myState`.
 - `write(bool state)` sätter `myState = state`.
 - `toggle()` togglar `myState`.

För denna klass ska copy- och move-konstruktorerna samt motsvarande operatorer raderas:

- Kopieringskonstruktor.
- Förflyttningskonstruktor.
- Kopieringstilldelningsoperator.
- Förflyttningstilldelningsoperator.

Viktigt:

- De överlagrade metoderna ska markeras `override`.
 - Klassen ska inte heller kunna ärvas vidare (klassen ska markeras `final`).
-

3. Factory (4p)

Betrakta nedanstående tomma stubb-factory:

```
namespace driver::factory
{
class Stub {};
} // namespace driver::factory
```

Du ska göra följande:

- Lägg till en metod `gpio(std::uint8_t pin)`, som returnerar en smart pekare till en GPIO-stubb, omvandlad till motsvarande GPIO-interface (`std::unique_ptr<driver::gpio::Interface>`).
- Ignorera explicit angivet pin-nummer i denna implementation, eftersom stubben inte använder den.
- Inkludera headerfiler för att använda smarta pekare samt `std::uint8_t`.
- Undvik råa pekare i denna implementation; skapa unika pekare enligt modern C++-stil.

OBS! Konstruktorer, destruktorn och kopierings- och förflyttningssemantik kan skippas.

VG-uppgifter

4. Klasstemplate (6p)

Skapa ett klasstemplate för ett stubb-EEPROM `driver::eeprom::Stub<std::uint16_t Size>`:

- Denna klass ska ärvä ett interface med namnet `driver::eeprom::Interface`, som inte behöver implementeras (anta att denna existerar).
- Anta att interfacet innehåller följande metoder:
 - `read(std::uint16_t address)`
 - `write(std::uint16_t address, std::uint8_t data)`

För denna klass gäller följande:

- Template-parameter `Size` utgör storleken på EEPROM-minnet i byte.
- Som default ska EEPROM-minnet sättas till `1024` byte.
- Metoderna ska markeras `override` och `noexcept`.

Denna klass ska innehålla:

- En constraint via en `static_assert` så att EEPROM-minnets storlek inte kan sättas till `0`. Om så är fallet ska felmeddelandet `EEPROM size must be greater than 0!` skrivas ut vid kompilering.
- En privat medlemsvariabel `myData`, som utgör en statiskt allokerad bytearray av angiven storlek `Size` (`std::uint8_t myData[Size]`).
- En default-konstruktor, som initierar arrayen med nollor (`{}`).
- En destruktor, som sätts till `= default`.
- `read(std::uint16_t address)`:
 - Returnerar innehållet (en byte) på angiven adress om adressen är giltig (`address < Size`), annars `0U`.
 - Ska vara `const`.
- `write(std::uint16_t address, std::uint8_t data)`:
 - Skriver en byte till angiven adress i minnet om adressen är giltig (`address < Size`).
 - Om skrivning sker returneras `true`, annars returneras `false`.

För denna klass ska copy- och move-konstruktorerna samt motsvarande operatorer raderas.

Klassen ska inte heller kunna ärvas vidare.

Valfritt (inför uppgift 5):

- Lägg också till motsvarande factory-metod `eeprom()`, som returnerar en smart pekare till en EEPROM-stubb, omvandlad till motsvarande EEPROM-interface (`std::unique_ptr<driver::eeprom::Interface>`):
 - Använd default-storleken `1024` byte vid skapande av EEPROM-stubben genom att lägga till `<>` efter att ha angett typen `driver::eeprom::Stub`.
-

5. Flertrådat testprogram (6p)

Betrakta koden nedan, som använder sig av delarna skapade i del 1 - 4.

Anta att:

- Samtliga interfaces och factory-metoder är korrekt implementerade.
- Nödvändiga inkluderingsdirektiv är tillagda.

```
namespace
{
    struct Mock
    {
        Mock() noexcept;
        void runTest(std::uint8_t pressCount = 5U) noexcept;

    private:
        void wait_ms(std::uint16_t ms) noexcept;
        void simulateButtonEvent(std::uint8_t pressCount) noexcept;
        void detectButtonPress(bool& buttonPressed) noexcept;
        void toggleLed(const bool& buttonPressed) noexcept;

        bool myStopFlag;
        std::unique_ptr<driver::gpio::Interface> myButton;
        std::unique_ptr<driver::gpio::Interface> myLed;
        std::unique_ptr<driver::eeprom::Interface> myEeprom;
    };

    // -----
    Mock::Mock() noexcept
        : myStopFlag{false}
        , myButton{nullptr}
        , myLed{nullptr}
        , myEeprom{nullptr}
    {
        constexpr std::uint8_t mockPin{0U};
        driver::factory::Stub factory{};
        myLed = factory.gpio(mockPin);
        myButton = factory.gpio(mockPin);
        myEeprom = factory.eeprom();
    }

    // -----
    void Mock::runTest(const std::uint8_t pressCount) noexcept
    {
        bool buttonPressed{false};

        std::thread t1{&Mock::simulateButtonEvent, this, pressCount};
        std::thread t2{&Mock::detectButtonPress, this, std::ref(buttonPressed)};
        std::thread t3{&Mock::toggleLed, this, std::cref(buttonPressed)};

        t1.join();
```

```

    t2.join();
    t3.join();
}

// -----
void Mock::wait_ms(const std::uint16_t ms) noexcept
{
    std::this_thread::sleep_for(std::chrono::milliseconds(ms));
}

// -----
void Mock::simulateButtonEvent(const std::uint8_t pressCount) noexcept
{
    constexpr std::uint16_t pollInterval_ms{100U};
    const std::size_t eventCount{2U * pressCount};

    for (std::size_t i{}; i < eventCount; ++i)
    {
        myButton->toggle();
        wait_ms(pollInterval_ms);
    }
    myStopFlag = true;
}

// -----
void Mock::detectButtonPress(bool& buttonPressed) noexcept
{
    constexpr std::uint16_t pollInterval_ms{10U};
    bool buttonPrev{false};

    while (!myStopFlag)
    {
        const bool buttonCurrent{myButton->read()};
        buttonPressed = buttonCurrent && !buttonPrev;
        buttonPrev    = buttonCurrent;
        wait_ms(pollInterval_ms);
    }
}

// -----
void Mock::toggleLed(const bool& buttonPressed) noexcept
{
    constexpr std::uint16_t pollInterval_ms{10U};
    constexpr std::uint16_t ledAddress{0U};

    const bool state{static_cast<bool>(myEeprom->read(ledAddress))};
    myLed->write(state);

    while (!myStopFlag)
    {
        if (buttonPressed)
        {
            myLed->toggle();
            const bool state{myLed->read()};

```

```
        myEeprom->write(ledAddress, static_cast<std::uint8_t>(state));
        const char* stateStr{myLed->read() ? "on" : "off"};
        std::cout << "Button pressed: the LED is " << stateStr << "!\n";
    }
    wait_ms(pollInterval_ms);
}
} // namespace

// -----
int main()
{
    Mock mock{};
    mock.runTest();
    return 0;
}
```

Svara på följande frågor:

- Vad sker i koden?
 - Koden är inte trådsäker:
 - Förklara varför.
 - Föreslå minst två konkreta kodändringar.
-